

Notch Activities

An activity is a source-defined unit of notch content. It gives the app a stable identifier, navigation metadata, SwiftUI presentations, availability and active state, and optional lifecycle or configuration behavior.

Activities are ordinary Swift types compiled into the app. The registry is not a runtime plugin loader and does not load third-party executable code.

Core types

NotchActivity uses associated view types, so activity implementations return concrete SwiftUI views. **AnyNotchActivity** performs type erasure only when different activity types are stored together in **ActivityRegistry**.

Every activity provides:

- A stable **ActivityID**. Do not derive it from a display name or change it after release.
- **ActivityMetadata** containing its name, SF Symbol, tint, and optional expanded height.
- An expanded presentation.
- **isAvailable**, which determines whether it appears in activity navigation.
- **isActive**, which represents ongoing work such as a running timer. It is independent of the currently selected page.

Compact presentation, full closed-notch live presentation, minimal closed-notch live presentation, configuration, and appearance lifecycle callbacks are optional.

When a feature should be an Activity

Use a **NotchActivity** when a feature has its own meaningful expanded experience and navigation identity, usually with state, lifecycle behavior, or optional configuration of its own. Pomodoro belongs in the Activity architecture because it has a dedicated expanded interface, persistent session state, timing behavior, and user-configurable cycles.

Minor controls, status indicators, transient notifications, overlays, and internal implementation details should not automatically become activities. Those should remain part of their owning feature or the existing closed-notch/HUD infrastructure.

Registration and navigation

Production activities are registered in the source-defined builder in **ActivityRegistry.shared**:

```
return try ActivityRegistry {  
    CalendarActivity()  
    MyActivity()  
}
```

Use namespaced permanent identifiers. Built-in activities use `builtin.<activity>` and community activities use `community.<publisher>.<activity>`. For example, Pomodoro uses `builtin.pomodoro`. Never rename an identifier after release.

Registration order controls activity order between Home and the legacy Activities page. Duplicate IDs prevent registry creation. An activity whose `isAvailable` value is false remains registered but is removed from tabs, pagination, and swipe navigation. If the selected activity becomes unavailable, navigation falls back to Home.

`ExampleActivity` intentionally is not in the production registry. Add it to the builder temporarily to see the example in the notch.

Expanded and compact presentations

The expanded host calls `makeExpandedView()`, applies `preferredExpandedHeight` when present, and invokes `activityDidAppear()` and `activityDidDisappear()` as the destination enters or leaves the hierarchy.

To declare compact content, return a concrete view from `makeCompactView()` and set `supportsCompactPresentation` to true. Compact content should be inexpensive and should only update as often as its displayed data requires.

```
var supportsCompactPresentation: Bool { true }

func makeCompactView() -> some View {
    Image(systemName: metadata.systemImage)
}
```

The closed-notch shell uses one live-provider stack for registered activity presentations, Timer, and Media. `makeCompactView()` is not inserted into that stack.

`makeCompactView()` remains a generic alternate presentation and is separate from the live-presentation API below. Pomodoro keeps its compact view as an example, but production closed-notch rendering uses its live presentation instead.

Closed-notch live presentations

Use live presentations for contextual, ongoing information that should appear around the closed notch. Activities publish visibility explicitly; `isActive` does not make a live presentation visible automatically.

Activities can provide two closed-notch live views:

- `makeLivePresentationView()` is the full live presentation. It is shown across both sides of the physical notch when this is the only selected live provider.
- `makeMinimalLivePresentationView()` is the minimal live presentation. It is shown on one side of the physical notch when the activity shares the closed notch with another live provider.

`makeMinimalLivePresentationView()` is additive. If an activity does not provide a dedicated minimal live view, the type-erased activity can reuse its existing live presentation. `makeCompactView()` remains

a separate generic alternate presentation and is not used as the minimal live presentation.

```
var livePresentationState: ActivityLivePresentationState {
    isRunning ? .visible(priority: .normal) : .hidden
}

func makeLivePresentationView() -> some View {
    RemainingTimeView()
}

func makeMinimalLivePresentationView() -> some View {
    RemainingTimeView()
}
```

The available priorities are `.low`, `.normal`, and `.high`. Priority remains part of `ActivityLivePresentationState` for source compatibility and explicit visibility metadata, but priority does not decide which live providers win the closed-notch stack. Recency decides selection.

The closed-notch activity stack behaves as follows:

- Zero eligible live providers: preserve the idle fallback behavior.
- One eligible live provider: render its full live presentation.
- Two or more eligible live providers: render the two most recently started providers using their minimal live presentations, one on each side of the physical notch.

A registered activity is adapted into a live provider and is eligible when `isAvailable == true` and `livePresentationState.priority != nil`. Timer and Media use non-navigation provider adapters over their existing managers. A provider becomes started when it transitions from not eligible to eligible. `ActivityLivePresentationCoordinator` subscribes to `LiveActivityPresentationProviderRegistry.objectWillChange` and reconciles eligibility centrally. Starts are not inferred from SwiftUI body evaluation, and the coordinator contains no provider-specific logic.

Recency is in memory for this iteration. If the app launches while activities are already eligible, the selector uses registry order as deterministic initial ordering. Hiding, ending, or making an activity unavailable removes it from current live selection and promotes the next most recent eligible activity. If an activity becomes eligible again later, it receives a new in-memory recency position.

The rendering boundary remains pure: it receives live providers plus the coordinator snapshot, filters eligible providers, sorts by recency, and returns no stack, a full stack, or a split stack. Startup, battery, Bluetooth, system HUDs, and bounded completion notifications remain transient interruptions outside the stack. Interruptions do not alter live selection or recency.

The core owns physical-notch spacing and default content dimensions. Providers, including adapted activities, may publish deterministic full and minimal presentation widths when their content needs less or more space. The selected stack is used for both rendering and closed-notch sizing so the two paths do not diverge. Each provider supplies accessory, full, and minimal content. Live views must not resize the notch or change navigation. Keep live content inexpensive: update only while its displayed data changes and derive elapsed time from timestamps rather than accumulated ticks.

Pomodoro publishes a normal-priority presentation with remaining time while running, a low-priority static presentation while paused, and hides it while ready or inactive. Timer is eligible while running or paused and ends when reset or finished. Media remains eligible through its configured post-pause grace period and ends when it becomes idle. Calendar remains an expanded-only registered activity and does not publish a live presentation.

State and lifecycle

Use SwiftUI state for presentation-local state:

- `@State` for values owned by one rendered view.
- `@StateObject` for a reference model owned by one rendered view.
- `@ObservedObject` for an existing shared manager.

Use a manager for persistent sessions, system observers, services, or state shared across displays. Do not move those responsibilities into the registry. Activity instances are shared, while the app can create a notch window per display.

Publish changes to `isActiveable`, `isActive`, and `livePresentationState`. Type erasure forwards `objectWillChange` to navigation and live-provider consumers. The coordinator owns in-memory live recency; providers only publish their own eligibility inputs and presentations.

Lifecycle callbacks are for work that genuinely follows visibility. They may run once per visible notch window, so implementations must be idempotent or reference-counted. Prefer normal SwiftUI `onAppear` and `onDisappear` inside activity views for view-local behavior.

Configuration

An activity with configuration returns a concrete configuration view and sets `supportsConfiguration` to true:

```
var supportsConfiguration: Bool { true }

func makeConfigurationView() -> some View {
    MyActivitySettings()
}
```

`ActivityConfigurationView(activityID:)` hosts that content. The registry does not automatically add a Settings sidebar entry; add an explicit entry when an activity needs user-facing configuration. This preserves the current Settings organization.

Creating an activity

1. Create an `ObservableObject` conforming to `NotchActivity`.
2. Choose a permanent ID and metadata.
3. Return the expanded SwiftUI view.
4. Add compact content, full live presentation, minimal live presentation, configuration, or lifecycle callbacks only when needed.

5. Register the activity in `ActivityRegistry.shared`.
6. Add the Swift file to the app target and add focused tests for ID, availability, state, and navigation behavior.
7. Build and run the macOS tests.

Minimal example:

```
@MainActor
final class WeatherActivity: NotchActivity {
    let id = ActivityID("community.weather")
    let metadata = ActivityMetadata(
        name: "Weather",
        systemImage: "cloud.sun.fill",
        tint: .blue
    )

    func makeExpandedView() -> some View {
        WeatherView()
    }
}
```

Keep activity initialization cheap. Managers that install observers, request permissions, or start polling should remain lazy and should only start when the feature requires them.